

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

Trabajo Práctico Integrador

Programación III

Food Store: Sistema de Gestion de Pedidos de Comida

Link al video de youtube: [Video defensa TPI - RUBIN](#)

Alexia Abi Rubin

Índice

1. Introducción
2. Objetivos del sistema
3. Tecnologías utilizadas
4. Estructura general del proyecto
5. Backend: modelo, persistencia y lógica de negocio
6. Frontend: autenticación, cliente y administración
7. Ejecución del proyecto
8. Relación con la rúbrica de evaluación
9. Conclusión

1. Introducción

El presente documento describe el Trabajo Práctico Integrador desarrollado para la materia Programación III. El proyecto se denomina Food Store y consiste en un sistema de gestión de pedidos de comida, compuesto por un backend desarrollado en Java con JPA/Hibernate y un frontend desarrollado con HTML, CSS, TypeScript y Vite.

La propuesta integra dos partes complementarias. Por un lado, el backend resuelve el modelo de datos, la persistencia, las operaciones CRUD, las consultas y la lógica de negocio mediante una aplicación de consola. Por otro lado, el frontend presenta una interfaz web que permite diferenciar el uso del sistema según el rol del usuario: cliente o administrador.

El objetivo general fue construir una solución funcional y organizada, respetando los requerimientos del TPI: modelo persistente, repositorios, validaciones, ABM de las entidades principales, pedidos, stock, autenticación, roles, catálogo, carrito, historial y panel administrativo.

2. Objetivos del sistema

El sistema Food Store fue pensado para representar el funcionamiento básico de una tienda de comida en la que los usuarios pueden consultar productos, armar pedidos y revisar su historial. A su vez, el administrador puede visualizar información general y gestionar los datos principales del sistema.

Objetivos funcionales principales:

- Permitir la gestión de categorías, productos, usuarios y pedidos.
- Registrar pedidos con control de stock y cálculo de totales.
- Aplicar baja lógica para evitar eliminaciones físicas de registros.
- Diferenciar funcionalidades según el rol del usuario.
- Permitir al cliente consultar catálogo, buscar productos, usar carrito y visualizar pedidos.
- Permitir al administrador acceder a dashboard, categorías, productos y pedidos.

Objetivos técnicos principales:

- Aplicar Programación Orientada a Objetos en el diseño del backend.
- Utilizar JPA/Hibernate para mapear entidades y relaciones.
- Implementar repositorios reutilizables y consultas JPQL.

- Mantener una estructura modular tanto en backend como en frontend.
- Usar TypeScript para organizar tipos, validaciones y funciones auxiliares en el frontend.

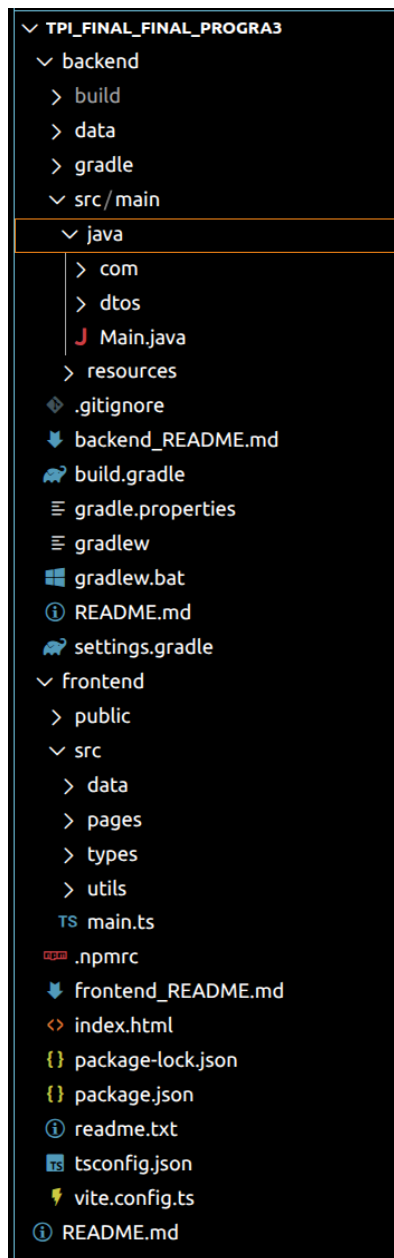
3. Tecnologías utilizadas

Área	Tecnologías / herramientas
Backend	Java, Gradle, JPA, Hibernate, H2 Database, Lombok
Frontend	HTML, CSS, TypeScript, Vite
Persistencia backend	Base de datos H2 administrada mediante JPA/Hibernate
Datos frontend	Archivos JSON y localStorage para sesión, carrito y datos de uso
Entrega	Repositorio comprimido en .zip, README, documentación y video mediante enlace público

4. Estructura general del proyecto

La entrega se organiza en una carpeta principal que contiene una carpeta para el backend, otra para el frontend, el README general y la documentación del proyecto. Esta separación permite identificar con claridad las responsabilidades de cada parte.

Estructura



En la entrega final no se incluyen carpetas generadas o dependencias instaladas, como node_modules, build o .gradle, ya que se regeneran al instalar dependencias o ejecutar los proyectos.

5. Backend: modelo, persistencia y lógica de negocio

El backend implementa la lógica principal del sistema mediante una aplicación Java ejecutada por consola. Su objetivo es resolver el modelo de datos persistente, las operaciones de administración y el comportamiento asociado a pedidos, stock y totales.

5.1 Modelo de datos

El modelo está compuesto por entidades JPA que representan los elementos centrales del dominio Food Store. Se utiliza una entidad base para concentrar atributos comunes y facilitar la reutilización dentro del modelo.

- Base: clase base con identificador y campos comunes para las entidades persistentes.
- Categoría: clasifica los productos disponibles en la tienda.
- Producto: representa los productos ofrecidos, con precio, stock y categoría asociada.
- Usuario: representa a los usuarios del sistema, incluyendo datos de acceso y rol.
- Pedido: representa la compra realizada por un usuario, con estado, fecha y total.
- DetallePedido: representa los ítems de cada pedido, incluyendo producto, cantidad, precio y subtotal.

5.2 Relaciones JPA

Las relaciones fueron modeladas para mantener coherencia entre las entidades y evitar duplicación innecesaria de información. Un producto pertenece a una categoría; un usuario puede tener varios pedidos; un pedido puede contener varios detalles; cada detalle se vincula con un producto.

Esta estructura permite consultar pedidos completos, calcular totales, actualizar stock y conservar información relevante de cada operación.

5.3 Persistencia y baja lógica

La persistencia se realiza mediante JPA/Hibernate sobre una base H2. Las operaciones principales permiten crear, listar, buscar, actualizar y dar de baja registros. La baja lógica evita eliminar físicamente los datos y permite conservar el historial del sistema.

Este criterio resulta importante especialmente para productos, categorías, usuarios y pedidos, porque mantiene la trazabilidad de las operaciones realizadas.

5.4 Repositorios y consultas

El backend utiliza una estructura de repositorios para separar el acceso a datos de la lógica de negocio. La implementación de BaseRepository permite reutilizar operaciones comunes y reducir repetición de código.

Además, se incorporan consultas JPQL para resolver búsquedas específicas, reportes y operaciones necesarias para el funcionamiento del sistema. El uso de Optional permite tratar resultados que pueden no existir, evitando errores por valores nulos y mejorando la validación de las operaciones.

5.5 Menú de consola y lógica de negocio

La consola funciona como punto de entrada del backend y permite probar las operaciones principales del sistema. Desde el menú se accede al ABM de categorías, productos y usuarios, además de la gestión de pedidos y reportes.

- Categorías: alta, listado, modificación y baja lógica.
- Productos: alta, listado, búsqueda, modificación, validaciones y baja lógica.
- Usuarios: alta, listado, búsqueda por email, modificación y baja lógica.
- Pedidos: creación de pedidos, asignación de usuario, carga de productos, control de stock, cálculo de subtotales y total.
- Estados: gestión del estado del pedido según el flujo definido.
- Reportes: consultas útiles para revisar información del sistema.

6. Frontend: autenticación, cliente y administración

El frontend implementa la interfaz web del sistema Food Store. Está desarrollado con TypeScript, HTML, CSS y Vite. La organización separa las vistas de autenticación, cliente y administración para mantener una estructura clara y fácil de recorrer.

6.1 Autenticación y roles

El sistema contempla login de usuarios, persistencia de sesión y diferenciación de roles. Según el rol autenticado, se habilita el acceso a las pantallas correspondientes.

- Usuario cliente: acceso al catálogo, carrito e historial de pedidos.
- Usuario administrador: acceso al panel administrativo, productos, categorías y pedidos.

- Sesión: persistencia mediante localStorage para conservar el estado de usuario durante el uso de la aplicación.

6.2 Funcionalidades del cliente

Desde el perfil cliente se puede navegar el catálogo de productos, realizar búsquedas, aplicar filtros y ordenar la información disponible. También se puede consultar el detalle de un producto antes de agregarlo al carrito.

El carrito conserva los productos seleccionados mediante localStorage, actualiza cantidades y permite calcular el total del pedido. Además, el cliente cuenta con una sección de historial para visualizar los pedidos realizados.

6.3 Funcionalidades de administración

El panel administrativo permite visualizar información general del sistema y acceder a las secciones principales de gestión. Incluye dashboard, categorías, productos y pedidos.

- Dashboard: muestra datos generales y estadísticas del sistema.
- Categorías: visualización y carga de categorías.
- Productos: visualización y carga de productos.
- Pedidos: visualización y administración de pedidos.

6.4 Datos del frontend

La carpeta public/data contiene archivos JSON utilizados como fuente de datos inicial para el frontend. Allí se incluyen datos de categorías, productos, usuarios y pedidos. Esta decisión permite probar la interfaz sin depender de un servidor externo durante la demostración.

- categorias.json
- productos.json
- usuarios.json
- pedidos.json

7. Ejecución del proyecto

Para ejecutar el backend, se debe ingresar a la carpeta backend y correr el wrapper de Gradle:

```
cd backend  
./gradlew run
```

En caso de ser necesario, se pueden otorgar permisos de ejecución al archivo gradlew:

```
chmod +x gradlew  
./gradlew run
```

Para ejecutar el frontend, se debe ingresar a la carpeta frontend, instalar dependencias y levantar el servidor de desarrollo:

```
cd frontend  
npm install  
npm run dev
```

El navegador debe abrir la dirección indicada por Vite, usualmente <http://localhost:5173>.

El video demostrativo se encuentra enlazado en el README general del proyecto, según el formato de entrega solicitado.

8. Relación con la rúbrica de evaluación

La siguiente tabla resume la relación entre los elementos desarrollados y los criterios principales de evaluación del TPI.

Criterio de evaluación	Elemento desarrollado en el proyecto
Backend - Modelo y persistencia	Entidades JPA, relaciones, persistencia con Hibernate/H2, CRUD y baja lógica.
Backend - Repositorios y JPQL	BaseRepository, repositorios específicos, consultas JPQL, Optional y validaciones.
Backend - Consola y negocio	Menú de consola, ABM de categorías, productos y usuarios, pedidos, stock, estados y totales.
Frontend - Autenticación y roles	Login, sesión persistente y diferenciación entre administrador y cliente.
Frontend - Cliente	Catálogo, búsqueda, filtros, ordenamiento, detalle, carrito e historial de pedidos.

Frontend - Administración	Dashboard, categorías, productos y pedidos desde vistas administrativas.
Calidad y entrega	Estructura separada en backend/frontend, README, documentación, video por enlace y zip sin dependencias generadas.

9. Conclusión

El desarrollo del Trabajo Práctico Integrador Food Store permitió integrar los contenidos principales de Programación III en un sistema completo, dividido en backend y frontend. En el backend se implementó un modelo de datos con entidades JPA, relaciones, repositorios, consultas, validaciones, operaciones CRUD, baja lógica, control de stock, estados de pedido y cálculo de totales. En el frontend se desarrolló una interfaz con autenticación, diferenciación de roles, catálogo, filtros, búsqueda, detalle de producto, carrito, historial de pedidos y panel de administración.

El proyecto cumple con una estructura clara y modular, permitiendo identificar la responsabilidad de cada parte. El backend concentra la lógica de negocio y la persistencia, mientras que el frontend organiza la experiencia de usuario para cliente y administrador.

Como mejora futura, se propone integrar ambas partes mediante una API REST, reemplazando el uso de archivos JSON del frontend por endpoints conectados directamente al backend. De esa manera, el sistema podría evolucionar hacia una aplicación completamente integrada, con persistencia centralizada y comunicación real entre interfaz y servidor.

En síntesis, Food Store representa una solución funcional para la gestión de pedidos de comida y demuestra la aplicación práctica de entidades, relaciones, persistencia, lógica de negocio, manejo de roles e interfaz web.